Company RAG Chatbot: a multi-tenant, bilingual retrieval-augmented assistant for company knowledge

Author: [YOUR FULL NAME] **ID:** [STUDENT ID] **Programme:** [PROGRAMME, UNIVERSITY] **Supervisor:** [SUPERVISOR NAME] **Date:** [MONTH YEAR]
Repository: [GITHUB REPOSITORY URL] **Walkthrough video:** [WALKTHROUGH VIDEO URL] **This report:** [TECHNICAL REPORT PDF URL]

1. Introduction

The **Company RAG Chatbot** is a multi-tenant web platform that lets employees ask questions about their own company's documents, in English and Kinyarwanda, and receive grounded natural-language answers from a large language model. It has three roles: the **SuperAdmin** (platform owner: global RAG settings, API keys, review of company requests, platform analytics), the **Company Admin** (employees, documents, start/pause/end of the service, analytics, exports) and the **Employee** (chat, ratings, reports). Stack: Next.js 14 and React 18, FastAPI (Python 3.12), PostgreSQL 16 with pgvector, Google Gemini (gemini-3.8-flash for answers, gemini-embedding-2 for embeddings), Docker. Sections: scope alignment (2), technical functionality (3), testing (4), performance and compatibility (5), code quality (6), limitations (7).

2. Scope alignment and feature validation

Planned features come from the project description, roadmap and the supervisor feedback. Result: **30 completed**, **4 partial**, **2 not implemented** (of 36 features). Path legend: B: = backend/app/, F: = frontend/src/app/, T: = backend/tests/.

Planned feature	Intended purpose	Status	Evidence (repository reference)
Global RAG configuration (documents, cleaning, chunking, embedding, retrieval, LLM, prompts, messages, chat limit)	One central, adjustable RAG setup for every company without code changes	Completed	B:routers/superadmin.py, B:schemas.py; UI F:superadmin/rag; T:smoke.py 'config update + activation'
Safe configuration changes (live validation, last working config kept, version history, rollback)	A bad key or model name must never break all companies	Completed	B:services/config_service.py; T:smoke.py 'invalid config rejected', 'NOT activated', 'previous config still active'
API credential management (add / edit / replace / delete, encrypted, never sent to browser)	Protect secrets while letting the admin change providers	Completed	B:security.py (Fernet), B:routers/superadmin.py; T:smoke.py 'key masked', 'never returns key'
Company management (register, view, activate/deactivate, enter as admin, per-company details)	Operate the multi-tenant platform	Completed	B:routers/superadmin.py; F:superadmin/page.tsx, F:superadmin/companies/[id]; T:smoke.py
Evaluation settings (ask every N answers; 1-5 rating and True/False questions)	Collect answer-quality feedback in a controlled way	Completed	B:routers/superadmin.py; F:superadmin/evaluation; T:smoke.py 'evaluation requested'
Platform and per-company analytics; reported cases	Monitor usage, quality, errors and complaints	Completed	B:services/analytics.py; F:superadmin/analytics; T:smoke.py 'platform analytics'
Employee management (create, edit, role, status, password reset)	Let each company control who can use the assistant	Completed	B:routers/company_admin.py; F:admin/employees; T:smoke.py 'employee created'
Document upload and processing pipeline (PDF, DOCX, TXT, MD; adjustable limits; progress; retry; EDA statistics)	Turn company files into a searchable knowledge base and show readiness	Completed	B:services/ingest.py, B:rag/*; F:admin/documents; T:smoke.py 'document processed', 'EDA stats stored'. Legacy .doc not supported
Chunking strategies incl. parent-child (plus recursive and fixed)	Better retrieval: search small chunks, answer with larger context	Completed	B:rag/chunking.py; T:test_language.py (4 chunking tests)
Hybrid retrieval: dense vector + sparse keyword search fused with RRF	Combine semantic and exact-term matching	Completed	B:rag/retrieval.py; db/schema.sql (HNSW + GIN indexes); benchmark Sec. 5
Reranking model and LLM-based query transformation (stated as 'later' in the proposal)	Higher answer precision	Partial	Extension slots and a follow-up query transform exist (B:rag/retrieval.py); no reranking model implemented
Start / Pause / End service with complete data deletion (tracked, retried)	Control usage and guarantee data removal	Completed	B:services/deletion.py, B:routers/company_admin.py; T:smoke.py 'deletion completed', v11 'deletion completed incl. anonymise'
Company analytics: feedback with question and answer, reports, frequent questions (count, info found), conversations, CSV downloads	Judge answer quality and see knowledge gaps	Completed	B:services/analytics.py; F:admin/analytics; T:smoke.py 'questions CSV download', 'conversation export'
Chat with follow-ups; maximum nine exchanges per chat enforced in the backend	Controlled conversations and cost	Completed	B:services/chat.py; db CHECK; T:smoke.py '10th question blocked by backend'
ChatGPT-style interface (New chat, chat history, Report)	Familiar, easy-to-use employee experience	Completed	F:chat/page.tsx
Processing indicator while an answer is prepared	Progress feedback without exposing model reasoning	Partial	Timed two-stage indicator in F:chat/page.tsx; not driven by real server progress (no streaming)
Rate answers 1-5 / True-False and report issues or recommendations	Quality feedback loop	Completed	F:chat/page.tsx; B:routers/chat.py; T:smoke.py 'feedback stored', 'report submitted'
Grounded answers, no-evidence behaviour, retrieval traceability	Avoid invented company information; let admins audit answers	Completed	B:services/chat.py, retrieval_logs table; T:smoke.py 'no-evidence question uses no-evidence path'
Distinct user-facing messages for AI failure, rate limit, document-processing failure	Clear errors while technical detail stays in protected logs	Completed	B:schemas.py (MessagesCfg), B:services/common.py (error_logs)
Background workers for ingestion and deletion	Long jobs must not block requests	Completed	B:worker.py (PostgreSQL queue, FOR UPDATE SKIP LOCKED); T:smoke.py (async ingestion)
Multi-tenant isolation (company_id on every record, backend scoping, row-level security)	No company can ever see another company's data	Completed	db/schema.sql (RLS); B:deps.py; T:smoke.py 'cannot retrieve company A's documents'; RLS test Sec. 4
Authentication, role-based access, Argon2 password hashing, audit logs	Protect accounts and administrative actions	Completed	B:security.py, B:deps.py, audit_logs; T:smoke_v11.py 'applicant cannot call admin / superadmin / chat APIs'
Backup and log-retention policy consistent with deletion	Deleted data must not linger	Partial	Policy stated in the privacy page; backups depend on the hosting provider; nothing implemented in code
English + Kinyarwanda: cleaning, chunking, embedding, retrieval, prompts	Serve bilingual companies; match a question in one language to a document in the other	Completed	B:rag/lang.py, chunking.py, clean.py, ai.py; Languages tab + live 'Language check'; T:test_language.py, smoke_v11.py (Kinyarwanda tests)

Planned feature	Intended purpose	Status	Evidence (repository reference)
Company request and review workflow (apply, approve / reject / request info, replies by portal and email)	Replace manual company registration	Completed	B:routers/apply.py, applications_admin.py; F:apply, F:superadmin/applications; T:smoke_v11.py (registration, review, reply, reject, reapply, approve, onboarding checks). Email is outbound only and needs SMTP
Manual verification only (no automatic legitimacy checks)	Keep access open to small businesses	Completed	By design: the reviewer decides in B:routers/applications_admin.py
Privacy and terms onboarding (3 pages, versioned acceptance), data export before ending, anonymised feedback, storage disclosure, employee-training advice	Informed consent and data protection	Completed	B:routers/apply.py, cms.py, company_admin.py (/export); T:smoke_v11.py 'terms', 'export', 'anonymised'
Marketing site and DB-managed content (landing page, onboarding pages, form fields, languages)	Attract customers; change content without code	Completed	F:page.tsx; B:routers/cms.py; T:smoke_v11.py (CMS checks)
API documentation and tooling (Swagger, Postman collection)	Testability and integration	Completed	/docs (Swagger), postman/company-rag.postman_collection.json (109 requests)
Model evaluation lab (SuperAdmin): retrieval/embedding and LLM metrics, baselines, threshold analysis	Measure each AI part separately and tune it with evidence	Completed	B:eval/*, B:routers/model_eval.py; F:superadmin/model-eval; T:test_eval_metrics.py (9), smoke_eval.py (40); docs/MODEL_EVALUATION_METRICS.md. Real-Gemini runs still to be recorded
Favicon and app icons	Professional, recognisable product in browser tabs and home screens	Completed	F:app/icon.svg, favicon.ico, apple-icon.png; browser test 'favicon asset is served'
Automated testing (unit, integration, REST, browser, load)	Validate behaviour and find defects	Completed	267 automated checks, 0 failing: unit 39, integration 167, REST 32, browser 29; API coverage 84%. docs/TESTING_REPORT.md, docs/testing/logs
Acceptance testing with 20+ participants (SUS)	Show real users can complete tasks and rate usability	Partial	Questionnaire, response template and scoring script ready (docs/testing/acceptance); no participants tested yet
Local deployment with Docker	Reproducible setup	Completed	docker-compose.yml, RUN_LOCALLY.md
Cloud deployment: backend on Render, frontend on Vercel	Public availability	Not implemented	Hosting on Render/Vercel not done yet. Deployment guide prepared; production mode and managed-database compatibility verified locally (ENVIRONMENT=production, non-superuser owner, full test suites passed). Update after deploying.
Billing per usage; image/screenshot understanding; marketing emails	Declared as future work by the supervisor	Not implemented	Out of scope for this version

Scope changes and their justification

Change	What changed	Justification
Company registration	Manual registration by the SuperAdmin became an online request-and-review workflow.	Supervisor feedback (points 2, 3): scales onboarding and keeps manual verification.
Company is created at terms acceptance	The company record is created when the applicant accepts the terms, not at approval.	No workspace exists before consent to the terms.
Job queue	A PostgreSQL queue (FOR UPDATE SKIP LOCKED) replaced a separate Redis/Celery broker.	One fewer service to run and pay for; sufficient for the expected load.
Document types	DOCX only (no legacy .doc).	Reliable text extraction with a pure-Python library.
Embeddings	768-dimension vectors; gemini-embedding-2 supported (no task_type, prefixes instead).	pgvector HNSW indexes are limited to 2000 dimensions; the model is multilingual.
Chat limit	Nine exchanges (1 question + 8 follow-ups), configurable down to 1.	Reconciles the project description (9) and the roadmap (8).
End of service	Employee accounts are deleted too; only de-identified scores are kept.	Privacy decision and supervisor point 4.
Processing indicator	Client-side timed stages instead of server streaming.	Satisfies the requirement at far lower complexity; streaming listed as future work.

3. Technical functionality

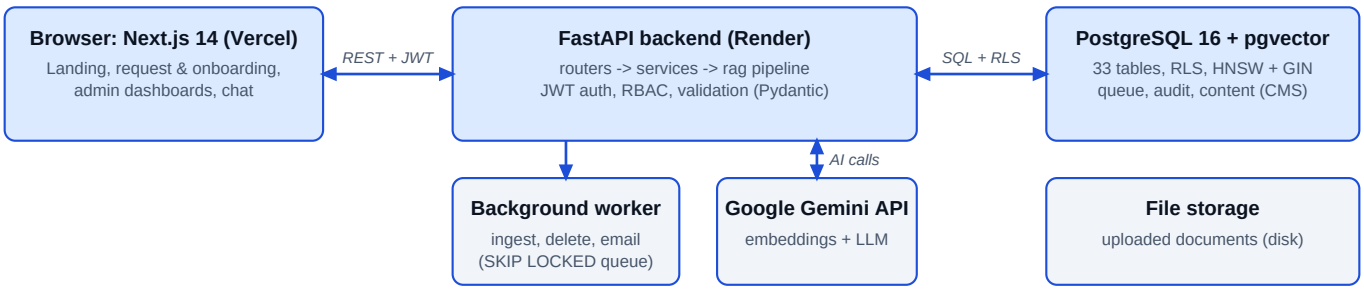


Figure 1. Architecture. The browser talks to the API only through REST + JWT; every request sets a row-level-security context in PostgreSQL.

Front-end / back-end interaction. The Next.js client calls a typed fetch wrapper (lib/api.ts) that adds the JWT and turns every failure into one error contract {code, message, fields}. Long jobs are polled (document progress every 2.5 s); downloads use authenticated blobs. Pydantic models validate all input, and form definitions, landing content, terms and languages are read from the database so they can change without a release.

Ingestion pipeline (background worker). Extract (pypdf, python-docx) -> clean (NFKC, unified apostrophes, hyphenation repair, repeated header/footer removal by line frequency across pages, soft-wrap merge) with EDA statistics -> language-aware chunking (parent-child, recursive or fixed; abbreviations, decimals and list numbers are protected; headings stay with their text; a chunk never mixes languages) -> batched embeddings with retry/back-off and pacing -> index (pgvector HNSW m=16, ef_construction=64; GIN full-text index). Jobs are claimed with FOR UPDATE SKIP LOCKED, so several workers cannot take the same job.

Question answering. Validate (service started, chat limit) -> detect language -> optional query transformation (short follow-ups get the previous question) -> embed -> **dense** search (cosine, HNSW) and **sparse** search (tsvector 'simple' + 'english', OR-query without stop-words) -> **Reciprocal Rank Fusion** (score = sum 1/(60 + rank)) -> evidence test (best dense similarity >= threshold) -> **parent expansion** (search small

chunks, answer with the larger parent) -> prompt with an explicit language directive -> LLM -> store question, answer, scores, tokens, latency and the retrieved chunks for later auditing. Without evidence, the configured no-evidence prompt is used so company facts are never invented. Vectors from different embedding models are never compared; changing the model re-queues all documents.

Multilingual support. Languages are database rows (detection words, sample sentences). A stop-word-ratio detector with a confidence margin tags each paragraph and each question; unsure cases fall back to 'answer in the language of the question'. A built-in *Language check* embeds a translation pair and an unrelated sentence and reports whether cross-language matching works for the configured models. Kinyarwanda strings were written by the developer and need native review.

Data handling and security. 33 tables; JSONB for versioned configuration blocks; immutable versions of the terms with per-user acceptance records; API keys encrypted (Fernet), passwords hashed (Argon2), login throttling, parameterised SQL, CSV-formula neutralisation in exports. Company isolation is enforced three times: company_id on every record, backend scoping from the authenticated user, and PostgreSQL row-level security (FORCE, so it also binds the table owner). 'End service' runs six tracked, retryable steps (anonymise scores, vectors, database rows incl. employee accounts, files, temp files, logs).

Company onboarding. Applicant account -> request form (fields stored in the database) -> SuperAdmin approves, rejects with a comment or asks for information; messages appear in the portal and are queued as emails -> three walkthrough pages with versioned acceptance -> the company is created and the applicant becomes its admin.

4. Testing and validation

Strategy	Why selected	How conducted	Results	Improvements made afterwards
Unit testing	Language logic is pure and easy to get wrong (bilingual detection, chunking, normalisation).	39 pytest tests in 3 files: language detection and chunking rules, cleaning, extraction errors, RRF fusion, query handling, security (Argon2, JWT expiry/tampering, key encryption, production guards), and every evaluation formula against hand-computed values.	39 / 39 passed; branch coverage of the core logic modules 86%.	The tests guard the abbreviation / heading rules of the splitter, the stop-word separation, and the refusal of weak production secrets.
Functional / integration (end-to-end API)	Most risk is in the interaction of API, database, worker and AI adapters, so tests drive the real HTTP API against PostgreSQL + pgvector.	smoke.py (51), smoke_v11.py (76), smoke_eval.py (40) and a 32-request REST script run complete journeys with deterministic test AI models; a Playwright suite (29 checks, Chromium, 23 screenshots) drives the real UI. Statement coverage with all suites: 84%.	228 / 228 passed.	Found and fixed: (1) commit-after-response race (4 of 30 create-then-login attempts failed) -> function-scoped DB dependency, fastapi>=0.121; (2) applicants wrongly treated as 'inactive company'; (3) 'Validate & activate' could freeze for about 2 minutes when Google is unreachable -> now fails after about 7 s per call.
Security and tenant-isolation testing	A multi-tenant platform fails badly if data leaks between companies.	Automated: cross-company reads (conversation, report, retrieval), role checks (403 'forbidden'), key masking. Manual database probes: with no company context the app role sees 0 rows; both suites also re-run in production mode connected as a non-superuser table owner.	All automated isolation and RBAC checks passed; row-level security applies even to the table owner (FORCE ROW LEVEL SECURITY).	Test helper uses an explicit RLS bypass so deletion checks are real, not vacuous.
Performance / load testing	To measure the system's own cost and find concurrency defects before users do.	tests/benchmark.py: login, 40 sequential chats, 10 simultaneous users, 0.92 MB ingestion, retrieval at 5,000 and 20,000 chunks (HNSW vs exact scan vs keyword search).	See Section 5.	Found: 10 simultaneous chats exhausted the 10-connection pool (PoolTimeout, errors). Fixed by not holding a pooled connection during slow AI calls; re-test: 0 errors. Regression test added.

4.2 Testing with different inputs and error handling

Input / situation	Expected behaviour	Observed
English question with evidence in documents	Grounded answer + source files	Passed: has_evidence true, sources listed
Kinyarwanda question; Kinyarwanda document	Answer in Kinyarwanda, evidence found	Passed: language 'rw', answer in Kinyarwanda
English question; Kinyarwanda document	Cross-language match	Passed with test model; live model: Language check (Sec. 4.3)
Question unrelated to documents	No-evidence answer, no invented facts	Passed: has_evidence false, no sources
10th question in one chat	Rejected by backend (limit 9)	Passed: 409 chat_limit_reached; counter capped at 9
Wrong password / duplicate email / weak password	Clear errors, nothing created	Passed: 401 / 409 / 422
.exe upload, duplicate file, oversize file	Rejected with reason	Passed (.exe and duplicate); size limit enforced in code
Outdated terms version submitted	Refused; latest version required	Passed: 422 must_accept
Company deactivated / service paused / ended	Access blocked, data kept (pause) or deleted (end)	Passed: 403 company_inactive / service_unavailable; deletion 6/6 steps
No / invalid API key when changing configuration	Config not activated, previous one stays active	Passed: 422 config_validation_failed. The user-facing ai_failure / rate_limit messages are implemented (B:services/chat.py) but not covered by an automated test

4.3 Live-model check (English / Kinyarwanda)

[PASTE THE RESULT OF SuperAdmin > RAG Configuration > Languages > Run check: models used, match vs unrelated similarity for each direction, whether the LLM answered in Kinyarwanda]

Test totals: **267 automated checks, all passing** (39 unit, 167 end-to-end integration, 32 REST, 29 browser); API statement coverage 84%, core-logic branch coverage 86%; plus the benchmark. Run: bash tests/run_all_tests.sh. Full evidence: docs/TESTING_REPORT.md.

5. Performance and compatibility

The benchmark uses deterministic test AI models, so it measures the platform's own cost (HTTP, authentication, retrieval, database, ingestion) and **excludes Google's response time**, which dominates what a user perceives in production. The left column is a 2-vCPU, 8 GB Linux test VM; the high-end column is filled by running tests/benchmark.py on the MacBook.

Measurement	Linux test VM (2 vCPU, 8 GB)	High-end: MacBook
Environment	Linux-6.18.44-fc-v242-x86_64, x86_64, 2 vCPU/cores, 7.8 GB RAM, Python 3.13.18	[run benchmark.py on the MacBook]
Login (Argon2 hashing)	p50 110 ms, p95 154 ms	[run benchmark.py on the MacBook]
Chat, 40 sequential questions	p50 153 ms, p95 209 ms, 0 errors	[run benchmark.py on the MacBook]
Chat, 10 simultaneous users	p50 786 ms, p95 1041 ms, 9.8 req/s, 0 errors	[run benchmark.py on the MacBook]

Measurement	Linux test VM (2 vCPU, 8 GB)	High-end: MacBook
Ingestion (test embedder)	0.92 MB -> 7,363 chunks in 16.1 s (457/s)	[run benchmark.py on the MacBook]
API process memory	281.4 MB idle -> 221.4 MB after load	[run benchmark.py on the MacBook]
Retrieval at 5,000 chunks	HNSW p50 1.8 / p95 2.7 ms; exact 1.7 ms; recall@10 0.71; keyword p50 4.9 ms	[run benchmark.py on the MacBook]
Retrieval at 20,000 chunks	HNSW p50 2.1 / p95 2.6 ms; exact 1.9 ms; recall@10 0.89; keyword p50 9.4 ms	[run benchmark.py on the MacBook]

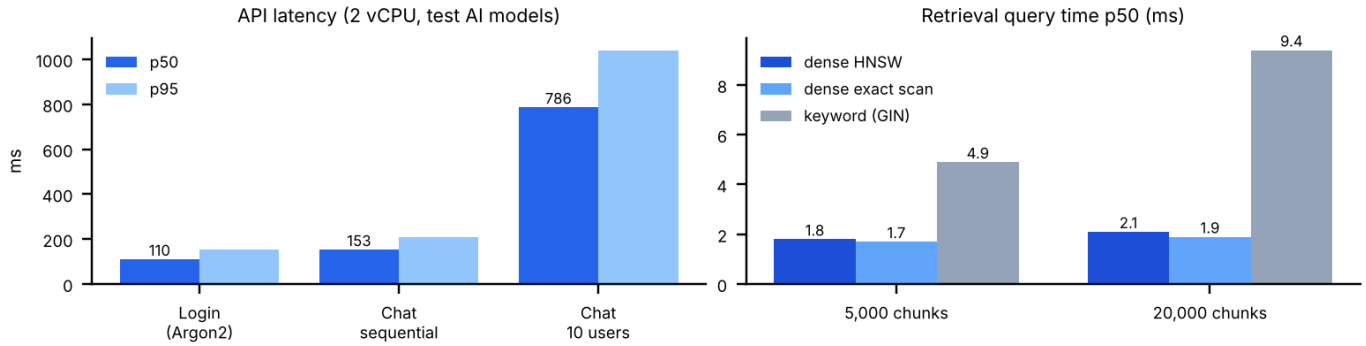


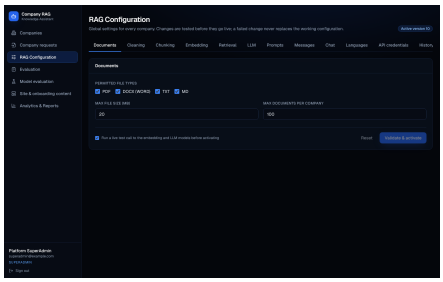
Figure 2. Measured on the Linux test VM (2 vCPU). HNSW and exact scan are equally fast at these sizes (a few milliseconds), so the index only pays off at larger scale and is kept for growth. Recall@10 of HNSW against exact search on random vectors (a worst case, no cluster structure) is 0.71 at 5,000 and 0.89 at 20,000 chunks with the default ef_search; raising ef_search trades speed for recall.

Front-end build (production): 21 routes, 87.3 kB shared JavaScript, 88-146 kB first-load per page; all pages are statically prerendered except two dynamic routes. Unit test runtime: about 1 s for 39 tests.

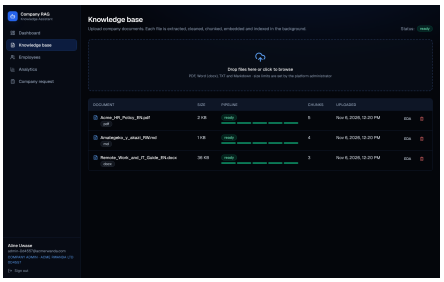
5.2 Software compatibility

Browser / OS	Version	Result	What was checked / notes
Google Chrome (latest) on macOS	[version]	[Pass/Fail]	[what you tested: login, chat, upload, analytics]
Safari on macOS	[version]	[Pass/Fail]	[notes]
Mozilla Firefox on macOS or Windows	[version]	[Pass/Fail]	[notes]
Microsoft Edge on Windows	[version]	[Pass/Fail]	[notes]
Mobile browser (phone, responsive layout)	[device + browser]	[Pass/Fail]	[notes]

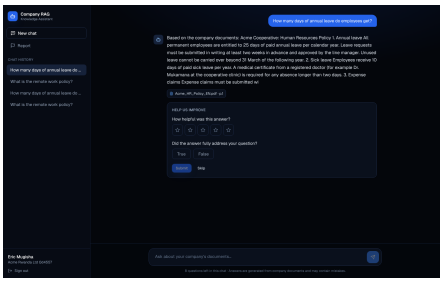
Figure 3. Key screens of the running application.



rag configuration



documents pipeline



employee chat

6. Code quality and technical implementation

Structure and separation of concerns. Backend: routers (HTTP only) -> services (business logic) -> rag/ (pipeline components) -> db helpers, with schemas, settings and security in their own modules; no monolithic script. Front-end: pages, shared UI primitives, layout shell, API client and auth context. Size: backend 4,379 lines, front-end 3,108, tests 1,500, SQL 870; 109 API operations; 33 tables.

OOP and SOLID, honestly stated. Encapsulation: Detector (language data and logic), provider classes keep keys and settings private. Abstraction and polymorphism: GoogleEmbedder / FakeEmbedder and GoogleLLM / FakeLLM expose the same methods (embed, generate) and are created by factories (get_embedder, get_llm), so the chat and ingestion code never depends on a vendor; the fake classes replace the real ones in every test (Liskov substitution). Inheritance: Pydantic configuration models, an exception hierarchy (AIError -> RateLimitError), ApiError in the client. Open/closed: new query transforms, providers and languages are added through registries, factories or database rows without editing callers. Dependency inversion and composition: FastAPI dependencies (tenant(), role_user()) inject the authenticated user and a row-level-security-scoped connection. The pipeline stages are plain functions by design; interfaces are duck-typed rather than formal abstract classes, which would be a natural next refinement.

Algorithms and problem solving. Reciprocal Rank Fusion; HNSW approximate nearest-neighbour search; parent-child retrieval; recursive splitter with overlap and boundary protection; stop-word-ratio language detection with margin; header/footer detection by cross-page line frequency; exponential back-off and rate-limit pacing; FOR UPDATE SKIP LOCKED work queue; Argon2 hashing and login throttling; connection-pool-aware request design (found by load testing).

Maintainability. Consistent naming (snake_case Python, camelCase TypeScript), docstrings on non-obvious modules, idempotent SQL migration verified against a fresh install (identical columns, indexes, constraints), configuration through environment variables, secrets never in the repository, README, RUN_LOCALLY.md, DATABASE.md, Swagger and a Postman collection.

7. Limitations and future work

Not done or limited: cloud deployment status as shown in Section 2; no OCR for scanned PDFs; no streaming of answers; no reranking model; email is outbound only; backups and retention rely on the hosting provider; login throttling and file storage are per instance (a multi-instance deployment needs shared storage and a shared counter); the free tier of the Gemini API may use submitted content to improve Google products, so a billing-enabled key is required for real company data; Kinyarwanda quality depends on the models and needs native review. Future work (supervisor): billing per usage, image understanding, marketing emails.